

## TAREA 5.3 - Desarrollo de una función factorial usando TDD



# Índice

|                                                    |           |
|----------------------------------------------------|-----------|
| <b>Objetivo</b>                                    | <b>3</b>  |
| <b>Enunciado</b>                                   | <b>3</b>  |
| <b>Desarrollo obligatorio con TDD</b>              | <b>4</b>  |
| ① Fase RED – Test que falla                        | 4         |
| ② Fase GREEN – Mínimo código para que pase el test | 5         |
| ③ Fase REFACTOR – Mejora del código                | 6         |
| <b>WebGrafía</b>                                   | <b>10</b> |

## Objetivo

Desarrollar en Python una función factorial(n) aplicando correctamente la técnica TDD (Test Driven Development), utilizando el framework pytest, y documentando visualmente cada paso del proceso.

El objetivo principal es demostrar que sabes trabajar con TDD, no solo que el resultado final funcione.

## Enunciado

Implementa la función factorial(n) cumpliendo los siguientes requisitos:

1. *factorial(0) debe devolver 1*
2. *factorial(1) debe devolver 1*
3. *factorial(5) debe devolver 120*
4. *Si n es negativo, la función debe lanzar ValueError*
5. *Si n no es un número entero, la función debe lanzar TypeError*

La función completa luego de realizar todos los pasos:



```
File Edit Selection ... < > Q Untitled (Workspace)
factorial.py X test_factorial.py
Desarrollo de Interfaces > tdd_factorial_Chica_Ángela > factorial.py
1  # función para calcular el factorial
2
3  def factorial(n):
4
5      # para ver si es entero
6      if not isinstance(n, int):
7          raise TypeError("no es entero")
8
9      # y para ver si es positivo o negativo
10     if n < 0:
11         raise ValueError("Es negativo")
12
13     if n == 0:
14         return 1
15     result = 1
16     for i in range(1, n + 1):
17         result *= i
18     return result
```

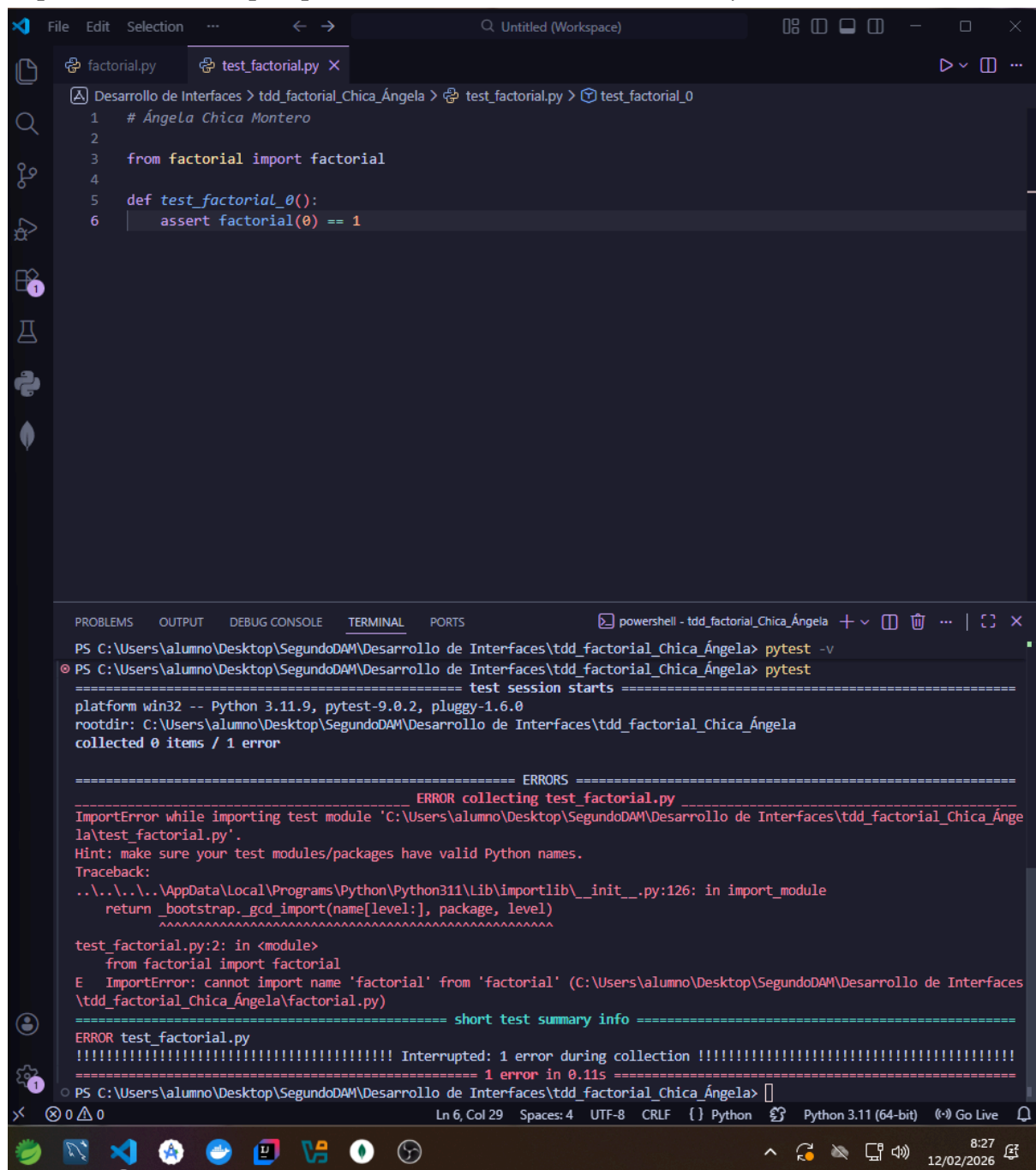
## Desarrollo obligatorio con TDD

Debes seguir estrictamente el ciclo TDD visto en clase.

### 1 Fase RED – Test que falla

- Escribe primero un test con pytest (por ejemplo, para `factorial(0)`).
- Ejecuta pytest.
- El test debe fallar.

El primer test falla porque aún no tenemos la función de *factorial*



The screenshot shows a Visual Studio Code editor with a file named `test_factorial.py` open. The file contains the following code:

```
1 # Ángela Chica Montero
2
3 from factorial import factorial
4
5 def test_factorial_0():
6     assert factorial(0) == 1
```

Below the editor, the terminal window shows the output of running `pytest` in a PowerShell session. The output indicates that the test session started successfully but failed due to an `ImportError` when trying to import the `factorial` module from `factorial.py`. The error message is:

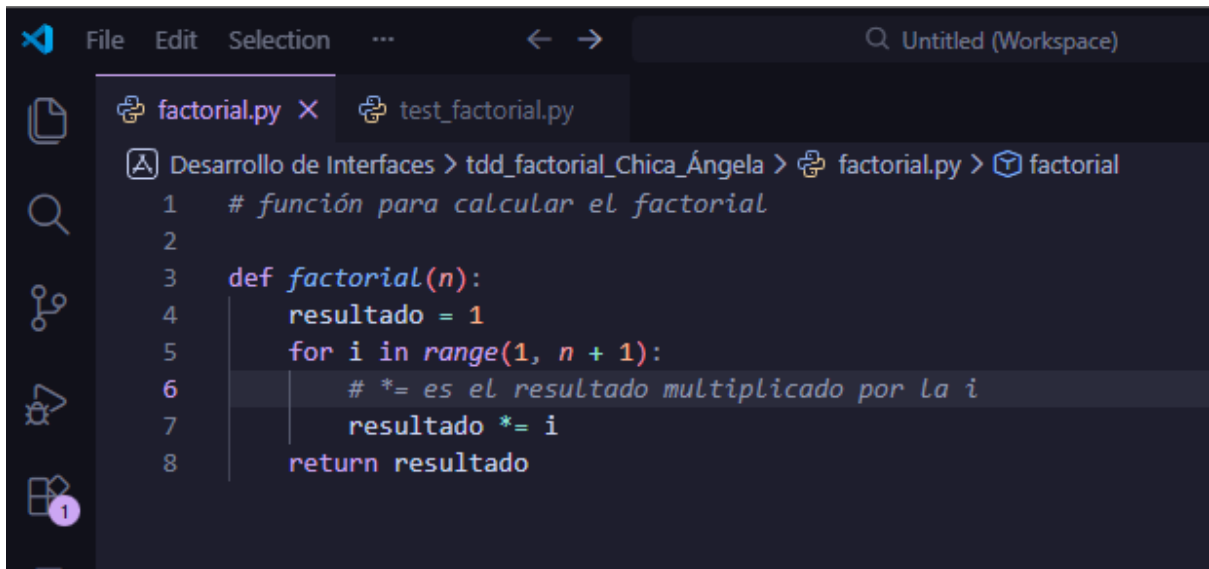
```
ImportError while importing test module 'C:\Users\alumno\Desktop\SegundoDAM\Desarrollo de Interfaces\tdd_factorial_Chica_Ángela\test_factorial.py'.
Hint: make sure your test modules/packages have valid Python names.
Traceback:
..\.venv\Lib\site-packages\importlib\_init_.py:126: in import_module
    return _bootstrap._gcd_import(name[level:], package, level)
test_factorial.py:3: in <module>
    from factorial import factorial
E ImportError: cannot import name 'factorial' from 'factorial' (C:\Users\alumno\Desktop\SegundoDAM\Desarrollo de Interfaces\tdd_factorial_Chica_Ángela\factorial.py)
```

The terminal also shows a summary of the test results, indicating that 1 error occurred during collection.

## ② Fase GREEN – Mínimo código para que pase el test

- Implementa el mínimo código posible para que el test pase.
- Ejecuta pytest de nuevo.

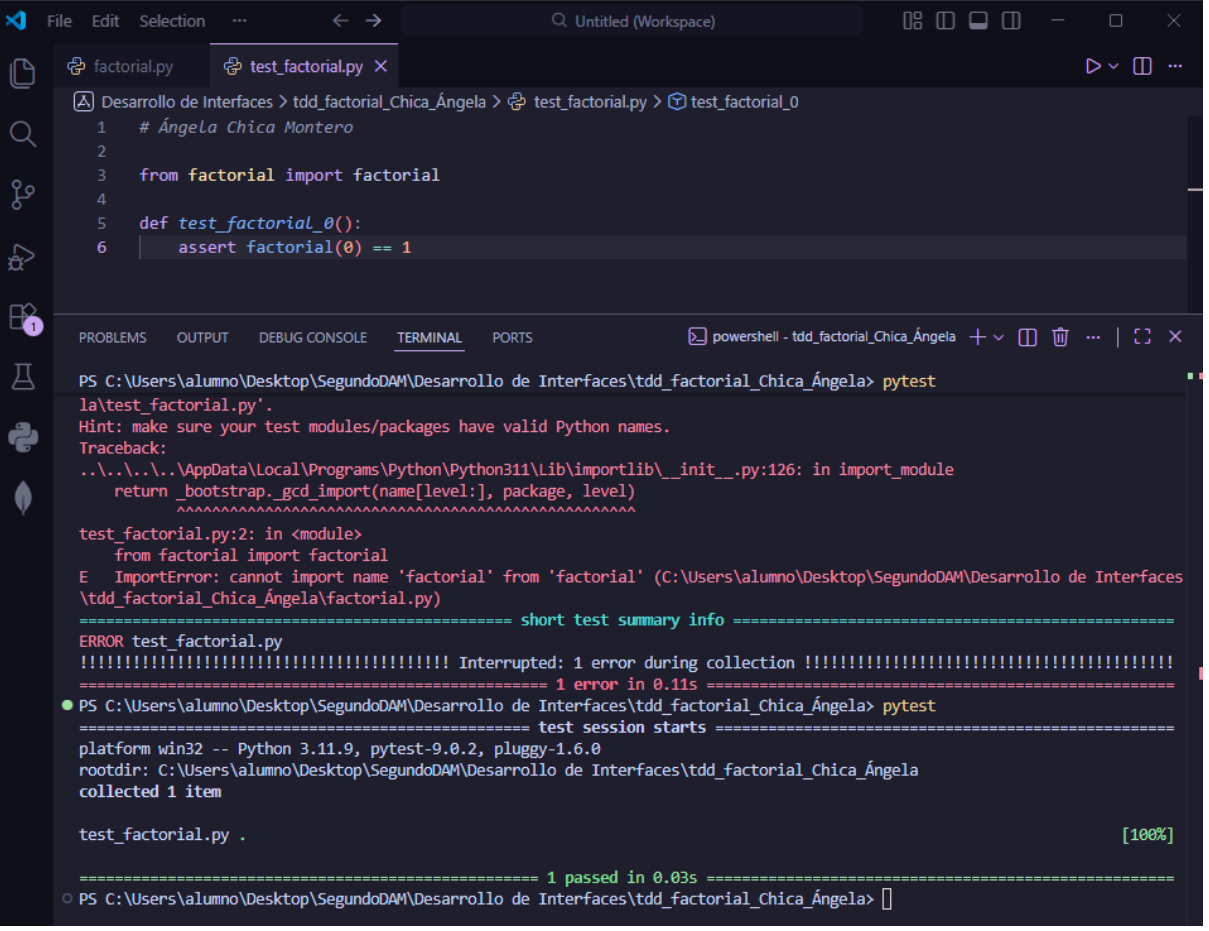
El código mínimo para que la función no falle es el siguiente y podemos para el test:



The screenshot shows a code editor with two tabs: `factorial.py` and `test_factorial.py`. The `factorial.py` tab is active, displaying the following Python code:

```
1  # función para calcular el factorial
2
3  def factorial(n):
4      resultado = 1
5      for i in range(1, n + 1):
6          # *= es el resultado multiplicado por la i
7          resultado *= i
8      return resultado
```

Ahora el test si funciona correctamente:



The screenshot shows a VS Code editor with a file named `test_factorial.py` open. The file contains the following code:

```
1 # Ángela Chica Montero
2
3 from factorial import factorial
4
5 def test_factorial_0():
6     assert factorial(0) == 1
```

The terminal window shows the output of running `pytest` in a PowerShell session. The output includes a traceback for an `ImportError` and a summary of the test results:

```
PS C:\Users\alumno\Desktop\SegundoDAM\Desarrollo de Interfaces\tdd_factorial_Chica_Ángela> pytest
la\test_factorial.py'.
Hint: make sure your test modules/packages have valid Python names.
Traceback:
..\..\..\AppData\Local\Programs\Python\Python311\Lib\importlib\_init_.py:126: in import_module
    return _bootstrap._gcd_import(name[level:], package, level)
test_factorial.py:2: in <module>
    from factorial import factorial
E ImportError: cannot import name 'factorial' from 'factorial' (C:\Users\alumno\Desktop\SegundoDAM\Desarrollo de Interfaces\tdd_factorial_Chica_Ángela\factorial.py)

===== short test summary info =====
ERROR test_factorial.py
!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!! Interrupted: 1 error during collection !!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!
===== 1 error in 0.11s =====

PS C:\Users\alumno\Desktop\SegundoDAM\Desarrollo de Interfaces\tdd_factorial_Chica_Ángela> pytest
platform win32 -- Python 3.11.9, pytest-9.0.2, pluggy-1.6.0
rootdir: C:\Users\alumno\Desktop\SegundoDAM\Desarrollo de Interfaces\tdd_factorial_Chica_Ángela
collected 1 item

test_factorial.py . [100%]

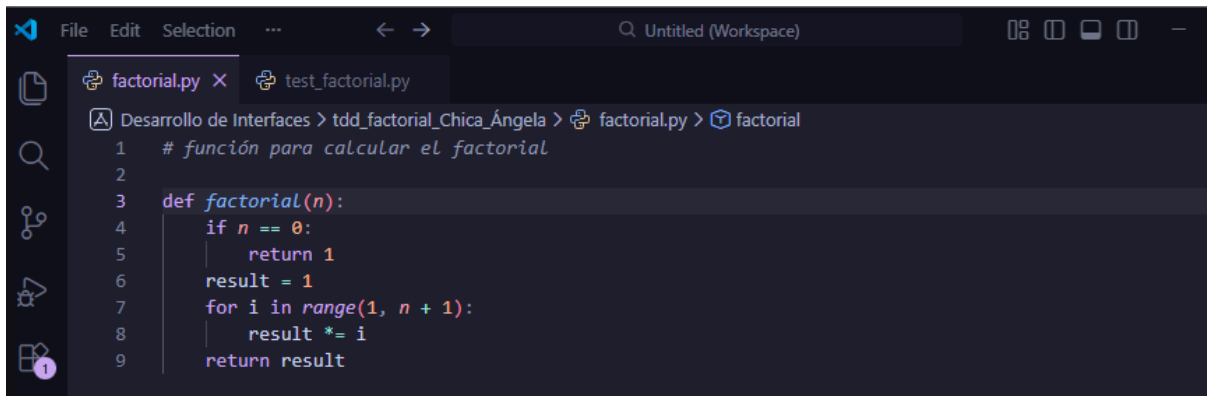
===== 1 passed in 0.03s =====

PS C:\Users\alumno\Desktop\SegundoDAM\Desarrollo de Interfaces\tdd_factorial_Chica_Ángela>
```

### ③ Fase REFACTOR – Mejora del código

- Añade nuevos tests de forma incremental:
  - `factorial(1)`
  - `factorial(5)`
  - Casos de error (negativo y no entero)
- Refactoriza la función para que quede clara y general.
- Los tests deben seguir pasando tras cada cambio.

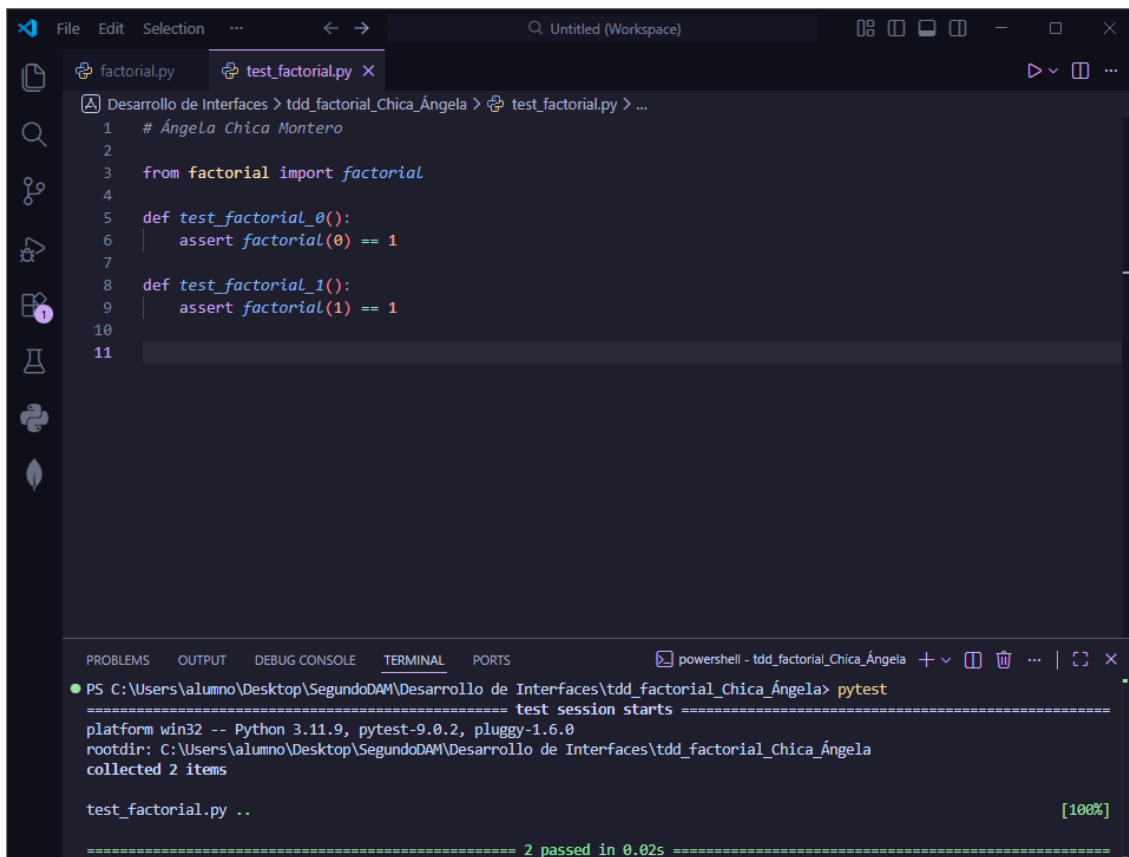
Para que la función vaya bien con los otros test debemos de modificarla y que puedan entrar valores aparte del 0:



A screenshot of the Visual Studio Code editor. The file explorer on the left shows a project structure: 'Desarrollo de Interfaces' > 'tdd\_factorial\_Chica\_Ángela' > 'factorial.py'. The editor window displays the code in 'factorial.py'.

```
1 # función para calcular el factorial
2
3 def factorial(n):
4     if n == 0:
5         return 1
6     result = 1
7     for i in range(1, n + 1):
8         result *= i
9     return result
```

Probamos primero el test con 1:



A screenshot of the Visual Studio Code editor. The file explorer shows 'test\_factorial.py' selected. The editor displays the test code. Below the editor, the terminal window shows the output of running 'pytest'.

```
1 # Ángela Chica Montero
2
3 from factorial import factorial
4
5 def test_factorial_0():
6     assert factorial(0) == 1
7
8 def test_factorial_1():
9     assert factorial(1) == 1
10
11
```

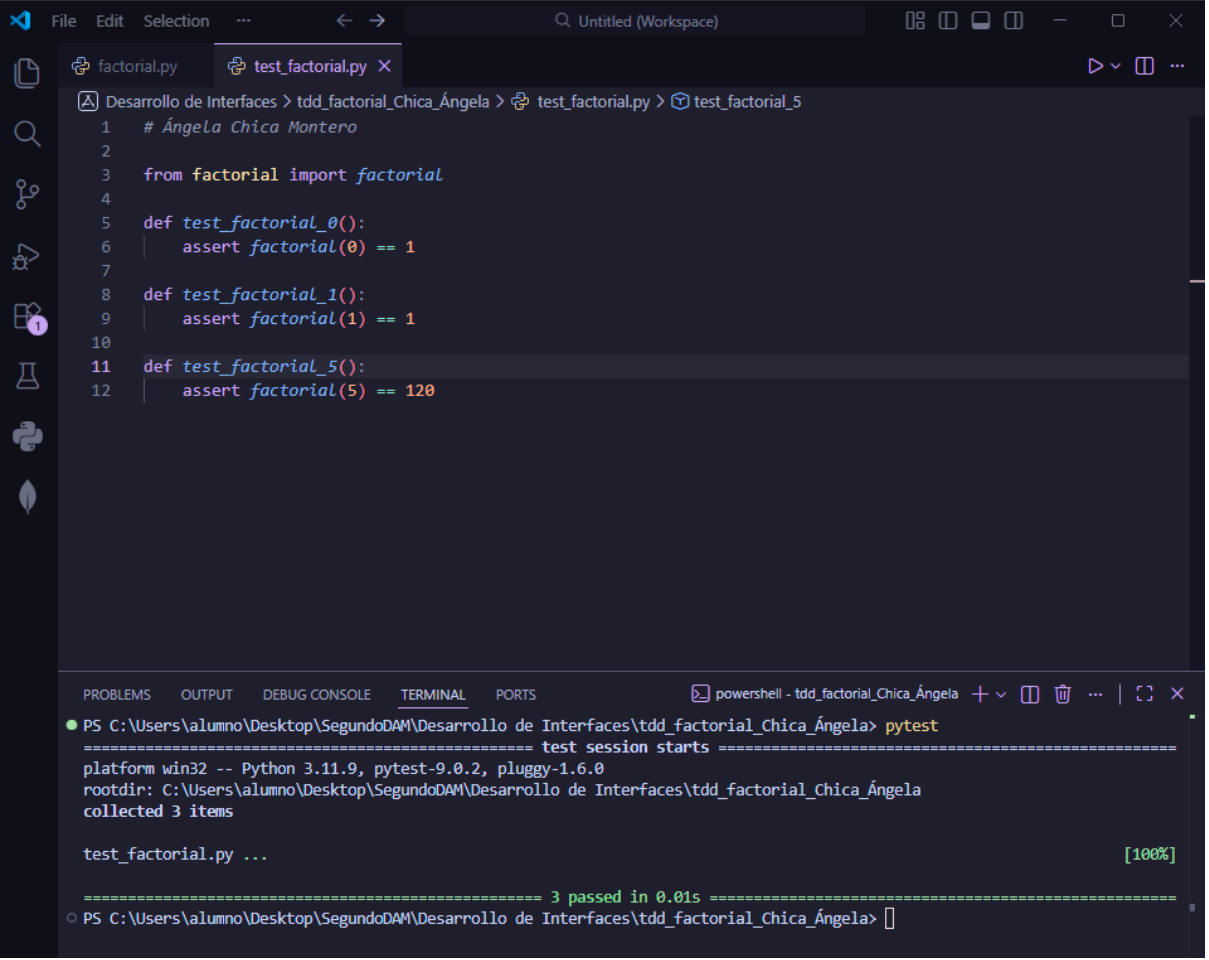
Terminal Output:

```
PS C:\Users\alumno\Desktop\SegundoDAM\Desarrollo de Interfaces\tdd_factorial_Chica_Ángela> pytest
===== test session starts =====
platform win32 -- Python 3.11.9, pytest-9.0.2, pluggy-1.6.0
rootdir: C:\Users\alumno\Desktop\SegundoDAM\Desarrollo de Interfaces\tdd_factorial_Chica_Ángela
collected 2 items

test_factorial.py .. [100%]

===== 2 passed in 0.02s =====
```

Ahora con el factorial de 5:



The screenshot shows a Visual Studio Code editor with two files open: `factorial.py` and `test_factorial.py`. The `test_factorial.py` file contains three test functions: `test_factorial_0()`, `test_factorial_1()`, and `test_factorial_5()`, each using `assert` to verify the output of the `factorial` function. The terminal at the bottom shows the command `pytest` being executed, which successfully runs all three tests, resulting in "3 passed in 0.01s".

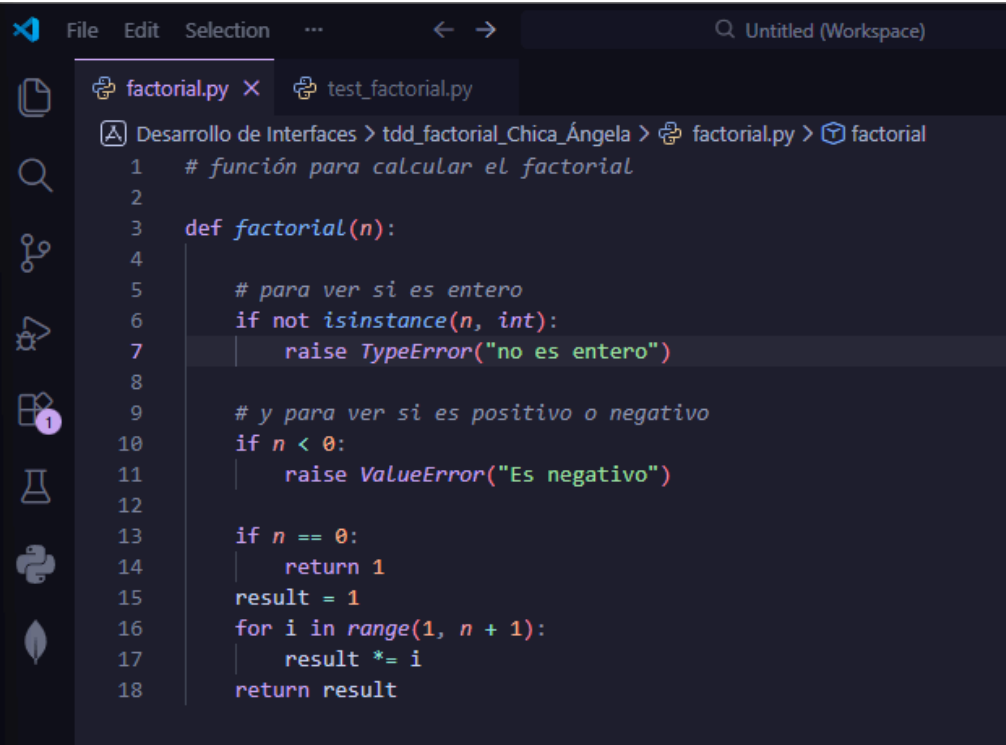
```
1 # Ángela Chica Montero
2
3 from factorial import factorial
4
5 def test_factorial_0():
6     assert factorial(0) == 1
7
8 def test_factorial_1():
9     assert factorial(1) == 1
10
11 def test_factorial_5():
12     assert factorial(5) == 120
```

```
PS C:\Users\alumno\Desktop\SegundoDAM\Desarrollo de Interfaces\tdd_factorial_Chica_Ángela> pytest
===== test session starts =====
platform win32 -- Python 3.11.9, pytest-9.0.2, pluggy-1.6.0
rootdir: C:\Users\alumno\Desktop\SegundoDAM\Desarrollo de Interfaces\tdd_factorial_Chica_Ángela
collected 3 items

test_factorial.py ... [100%]

===== 3 passed in 0.01s =====
PS C:\Users\alumno\Desktop\SegundoDAM\Desarrollo de Interfaces\tdd_factorial_Chica_Ángela>
```

Y para que los test funcionen con negativos y no enteros debemos de realizar las siguientes modificaciones en la función anterior:

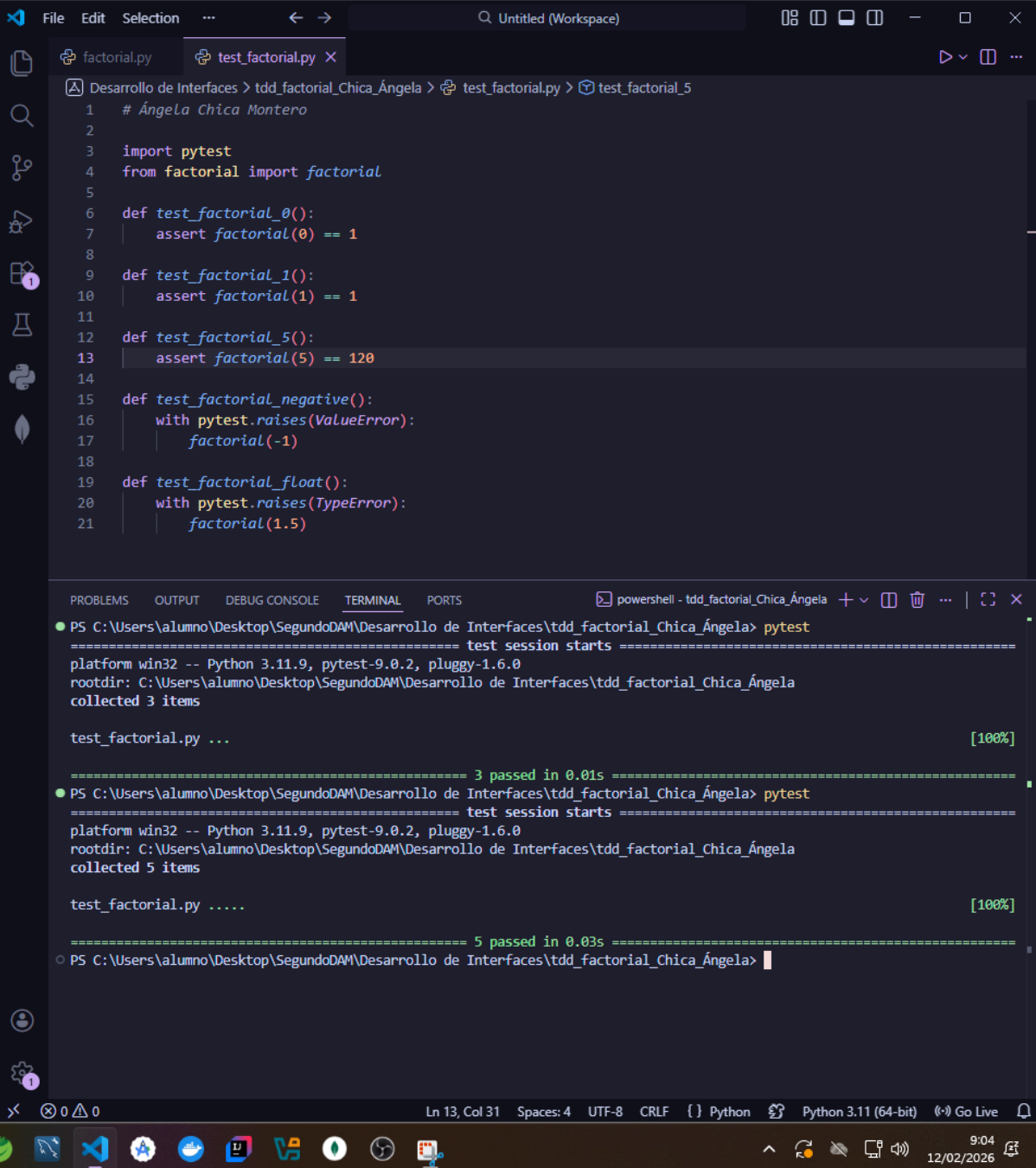


The screenshot shows the `factorial.py` file with updated logic. It includes checks for non-integer inputs using `isinstance` and for negative inputs using `n < 0`. The function still calculates the factorial for non-negative integers using a loop.

```
1 # función para calcular el factorial
2
3 def factorial(n):
4
5     # para ver si es entero
6     if not isinstance(n, int):
7         raise TypeError("no es entero")
8
9     # y para ver si es positivo o negativo
10    if n < 0:
11        raise ValueError("Es negativo")
12
13    if n == 0:
14        return 1
15    result = 1
16    for i in range(1, n + 1):
17        result *= i
18    return result
```

Lo siguiente





The image shows a Visual Studio Code editor window with a Python file named `test_factorial.py` open. The file contains several test functions using `pytest` to verify the `factorial` function. The terminal at the bottom shows the output of running `pytest` twice, first with 3 tests passing and then with 5 tests passing.

```
1 # Ángela Chica Montero
2
3 import pytest
4 from factorial import factorial
5
6 def test_factorial_0():
7     assert factorial(0) == 1
8
9 def test_factorial_1():
10    assert factorial(1) == 1
11
12 def test_factorial_5():
13    assert factorial(5) == 120
14
15 def test_factorial_negative():
16     with pytest.raises(ValueError):
17         factorial(-1)
18
19 def test_factorial_float():
20     with pytest.raises(TypeError):
21         factorial(1.5)
```

Terminal Output:

```
PS C:\Users\alumno\Desktop\SegundoDAM\Desarrollo de Interfaces\tdd_factorial_Chica_Ángela> pytest
===== test session starts =====
platform win32 -- Python 3.11.9, pytest-9.0.2, pluggy-1.6.0
rootdir: C:\Users\alumno\Desktop\SegundoDAM\Desarrollo de Interfaces\tdd_factorial_Chica_Ángela
collected 3 items

test_factorial.py ... [100%]

===== 3 passed in 0.01s =====
PS C:\Users\alumno\Desktop\SegundoDAM\Desarrollo de Interfaces\tdd_factorial_Chica_Ángela> pytest
===== test session starts =====
platform win32 -- Python 3.11.9, pytest-9.0.2, pluggy-1.6.0
rootdir: C:\Users\alumno\Desktop\SegundoDAM\Desarrollo de Interfaces\tdd_factorial_Chica_Ángela
collected 5 items

test_factorial.py ..... [100%]

===== 5 passed in 0.03s =====
PS C:\Users\alumno\Desktop\SegundoDAM\Desarrollo de Interfaces\tdd_factorial_Chica_Ángela>
```

## WebGrafía

Apuntes → <https://experts-deny-b9a.craft.me/y3eKOUPLrtEM6g>

Tarea guida →

<https://experts-deny-b9a.craft.me/y3eKOUPLrtEM6g/b/1D6BDE8D-54BC-4B30-9559-CF383846E398/Ejercicio-Resuelto-Aplicando-TDD>